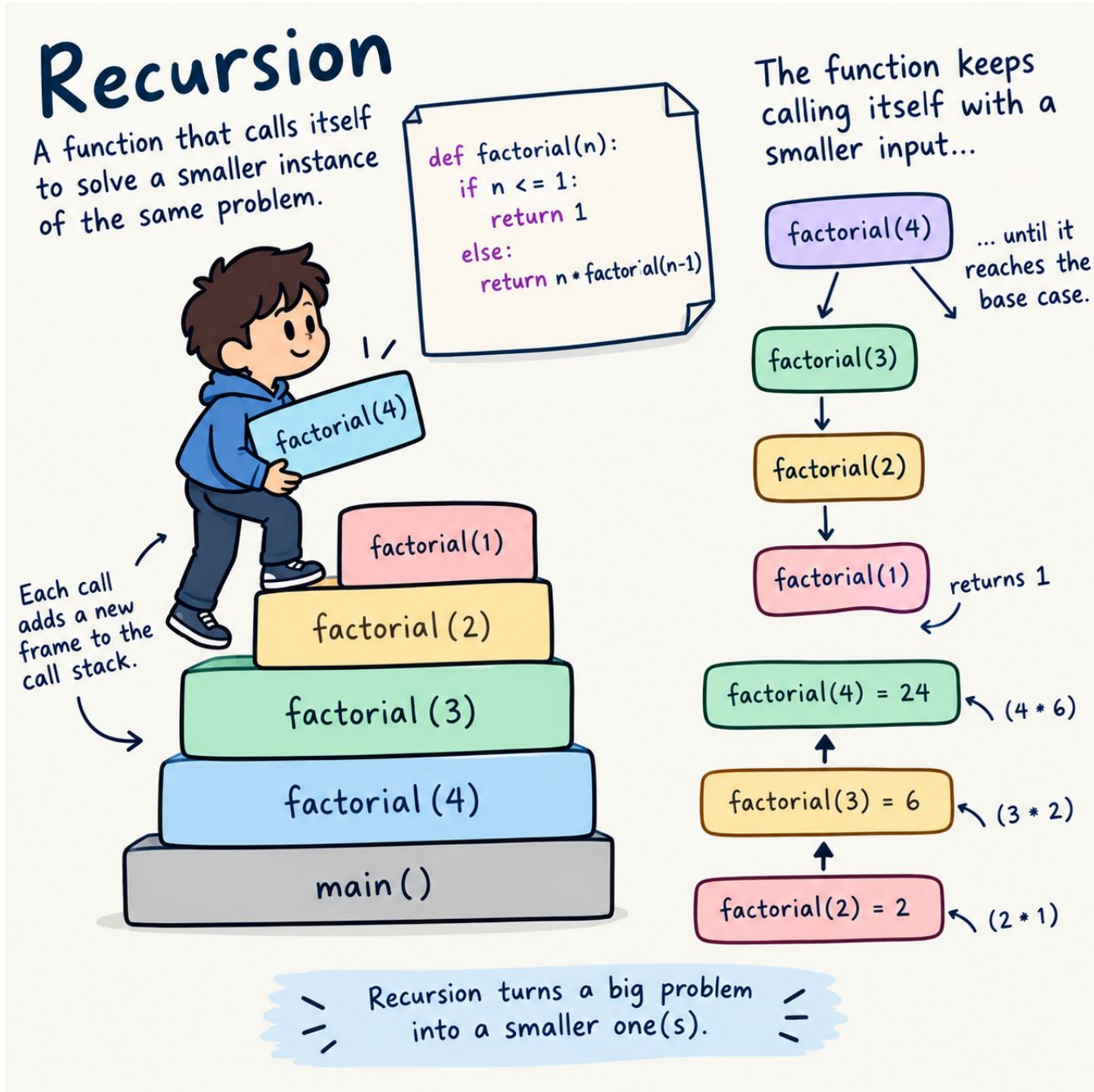
Algorithms & Data Structures

Week 04

Ahmet Feyzi Ateş

Recursion and Recursive Algorithms



Example 1: Recursive evaluation of *n*!

Definition: $n! = 1 \cdot 2 \cdot \ldots \cdot (n\text{-}1) \cdot n$ for $n \geq 1$ and $0! = 1$

Recursive definition of *n*!: $F(n) = F(n\text{-}1) \cdot n$ for $n \geq 1$ and $F(0) = 1$

ALGORITHM $F(n)$

```
//Computes n! recursively
//Input: A nonnegative integer n
//Output: The value of n!
if n = 0 return 1
else return F(n − 1) * n
```

Size: n (input numeric value)
Basic operation: multiplication or comparison
Recurrence relation:

Solving the recurrence relation

If we assume the input size is n, and basic operation is multiplication (M):

M(n) : # of multiplications needed to compute n!

M(*n*) = M(*n*-1) + 1, M(0) = 0

M(n) = M(n-2) + 1 + 1

= M(n-3) + 1 + 1 + 1

= M(n-4) + 1 + 1 + 1 + 1

= ……..

= ……..

= ……..

= M(0) + 1 + 1 + 1 + 1 + 1 +..... + 1

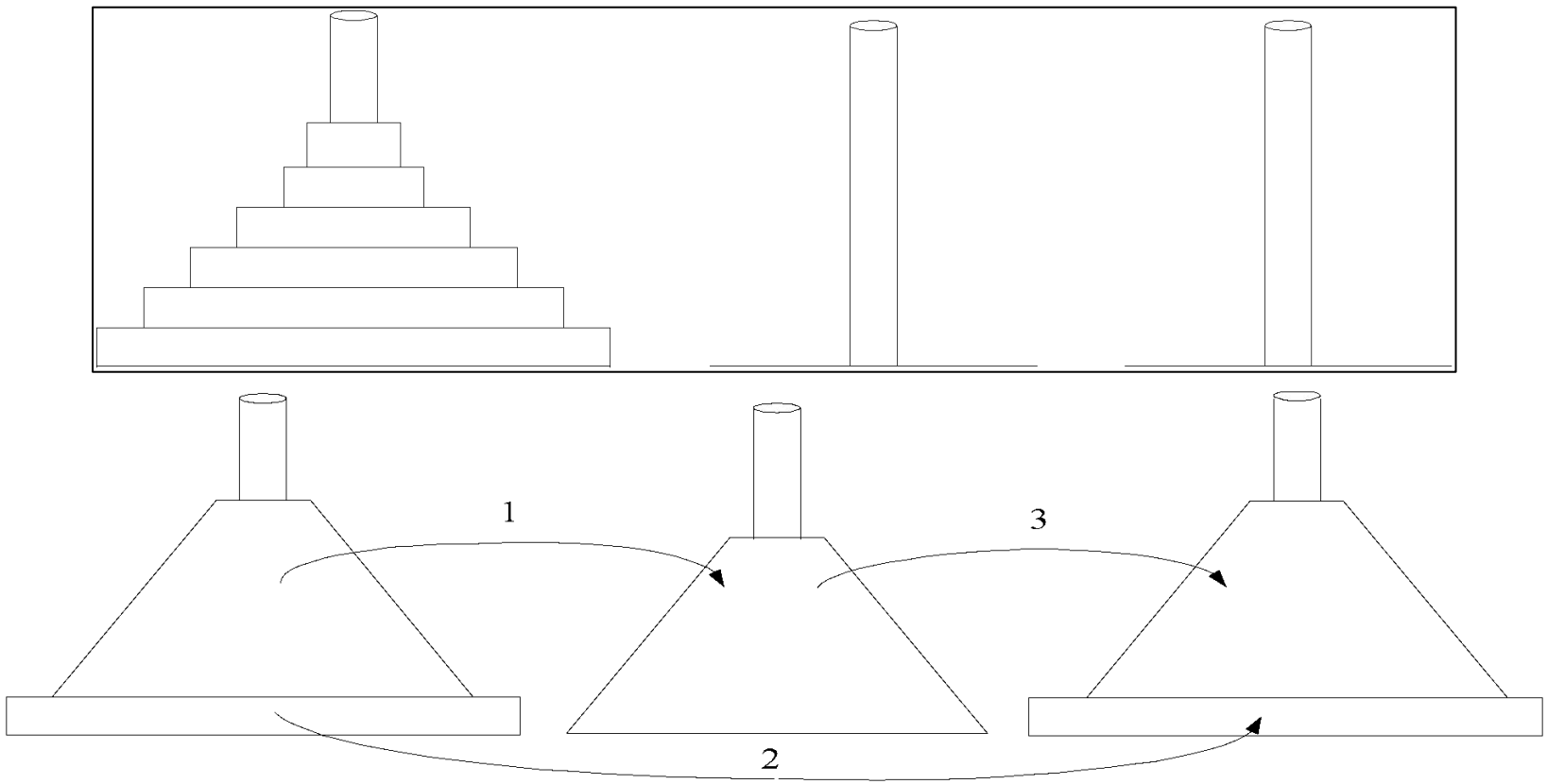
——----------

a total of n 1's here

= 0 + n

= n (Linear)

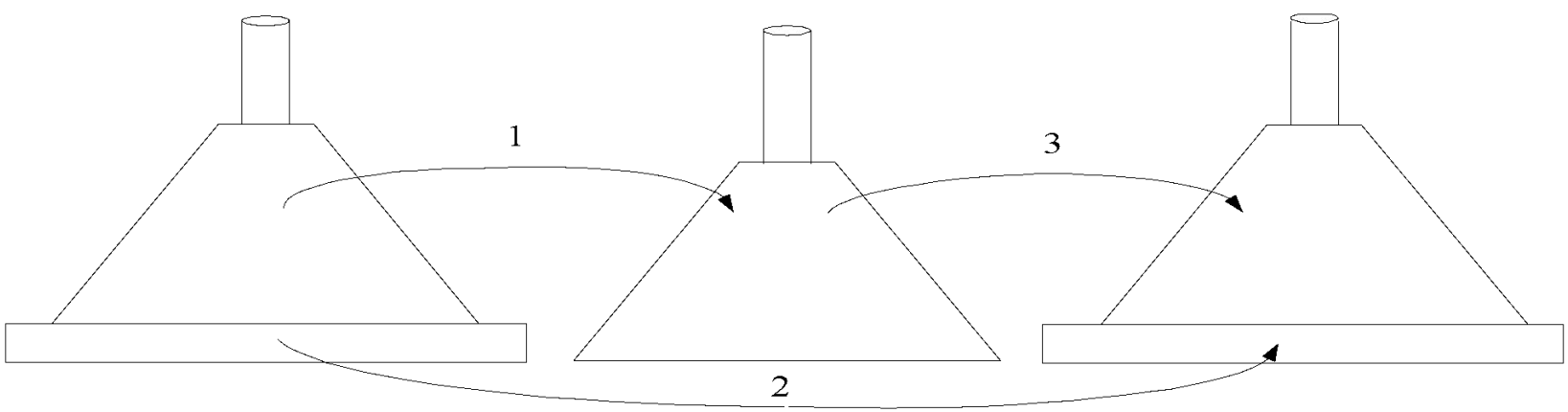
Example 2: The Tower of Hanoi Puzzle



- **Aim: Move the disks from the first peg to the third peg, with the help of second peg, without changing their order.**
- **Rules: You can move one disk at a time. • You can not place a larger disk on top of a smaller one**

Recursive Solution

- First move recursively n-1 disks from peg 1 to peg 2 (with peg 3 as auxiliary).
- Then move the largest disk directly from peg 1 to peg 3
- Finally, move recursively n-1 disks from peg2 to peg3 (using peg 1 as auxiliary).
- If n=1, simply move the single disk directly from the source peg to the destination peg.



Solving recurrence for number of moves

$$M(n) = M(n-1) + 1 + M(n-1), \qquad M(1) = 1$$

$$= 2M(n-1) + 1$$

$$= 2(2M(n-2)+1) + 1$$

$$= 2^2M(n-2) + 2 + 1$$

$$= 2^2(2M(n-3)+1) + 2 + 1$$

$$= 2^3M(n-3) + 2^2 + 2 + 1$$

.

.

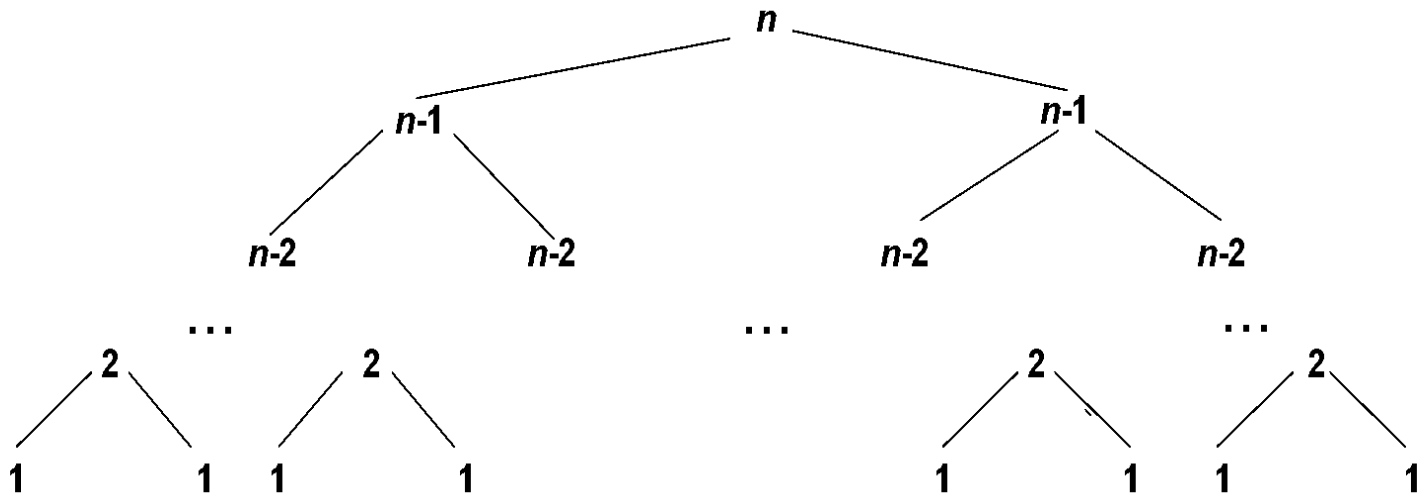
$$= 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \ldots\ldots + 1$$

$$= 2^i M(n-i) + 2^i - 1$$

$$= 2^{n-1} M(1) + 2^{n-1} - 1 \quad \text{(for } i = n-1\text{)}$$

$$= 2^n - 1 \qquad \text{(since } M(1) = 1\text{)}$$

Tree of calls for the Tower of Hanoi Puzzle



Example 2: Counting binary digits

Iterative version

ALGORITHM *Binary*(n)

```
//Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
count ← 1
while n > 1 do
    count ← count + 1
    n ← ⌊n/2⌋
return count
```

A. #of additions needed to find the number of bits in binary rep. of n)

Recursive version

ALGORITHM *BinRec*(n)

```
//Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
if n = 1 return 1
else return BinRec(⌊n/2⌋) + 1
```

Size: n (input numeric value)
Basic operation: addition or comparison
Recurrence relation:

Solving the recurrence relation

If we assume the input size is n, and basic operation is addition (A):

$$A(n) = A(\lfloor n/2 \rfloor) + 1, \quad A(1) = 0$$

$$A(n) = A(n/2) + 1$$
$$= A(n/4) + 1 + 1$$
$$= A(n/8) + 1 + 1 + 1$$
$$= \ldots\ldots\ldots\ldots\ldots\ldots$$
$$= \ldots\ldots\ldots\ldots\ldots\ldots$$
$$= \ldots\ldots\ldots\ldots\ldots\ldots$$
$$= A(1) + 1 + 1 + 1 + 1 + 1 + \ldots\ldots\ldots\ldots\ldots\ldots + 1$$

a total of logn 1's here

$$= 0 + \log n$$
$$= \log n \quad \text{(Logarithmic)}$$

Example 3: Recursive Binary Search

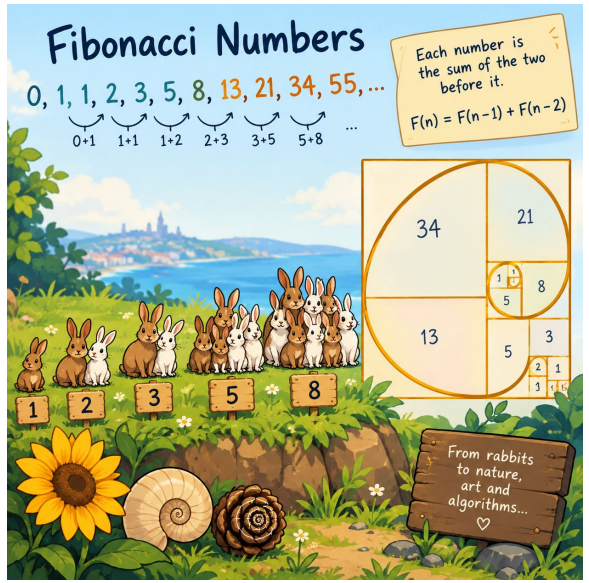
Algorithm *Recursive_Binary_Search* (A[0..n-1], low, high, key)

```
if low > high then
    return -1

mid ← ⌊(low + high) / 2)⌋
if A[mid] = key then
    return mid
else if key < A[mid] then
    return Recursive_Binary_SearchA, low, mid - 1, key)
else
    return Recursive_Binary_Search(A, mid + 1, high, key)
```

$$
\begin{aligned}
T(N) &\le T(N/2) + 1 && \text{[given]} \\
&\le T(N/4) + 1 + 1 && \text{[apply recurrence to first term]} \\
&\le T(N/8) + 1 + 1 + 1 && \text{[apply recurrence to first term]} \\
&\vdots \\
&\le T(N/N) + 1 + 1 + \ldots + 1 && \text{[stop applying, T(1) = 1]} \\
&= 1 + \lg N
\end{aligned}
$$

Example 4: Fibonacci numbers



The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, …

The Fibonacci numbers are a sequence F n of integers in which every number after the first two, 0 and 1, is the sum of the two preceding numbers. Hence the sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, …

Formally, they are defined by the Fibonacci recurrence:

$F(n) = F(n\text{-}1) + F(n\text{-}2)$

$F(0) = 0$

$F(1) = 1$

Alternative Solution with observation

$$F(n) = F(n-1) + F(n-2)$$

$$= F(n-2) + F(n-3) + F(n-3) + F(n-4)$$

$$= F(n-3) + F(n-4) + F(n-4) + F(n-5)$$

$$F(n-4) + F(n-5) + F(n-5) + F(n-6)$$

$$= F(n-4) + F(n-5) + F(n-5) + F(n-6) + F(n-5) + F(n-6) + F(n-6) + F(n-7) +$$

$$F(n-5) + F(n-6) + F(n-6) + F(n-7) + F(n-6) + F(n-7) + F(n-7) + F(n-8)$$

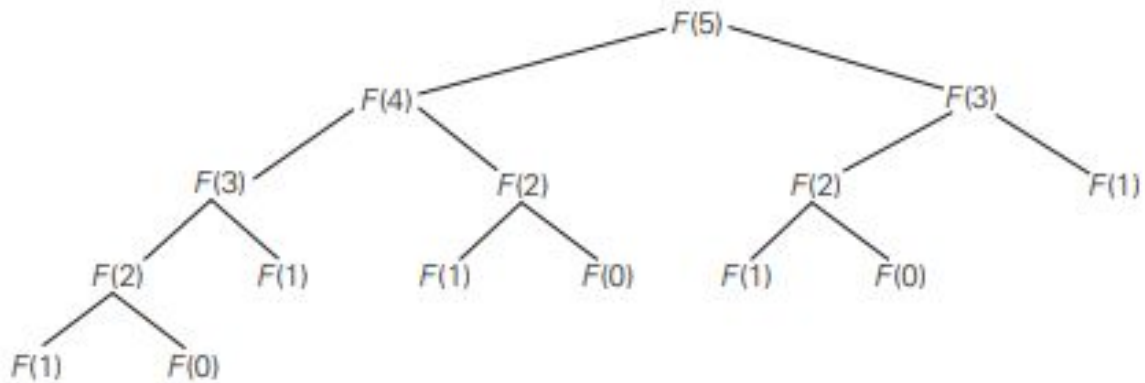
Quickly leads to exponential complexity!

Analysis by Observation

n	# of Fibonacci Calculations needed to find n^{th} fibonacci number.	2^n	φ^n ($\varphi \approx 1.61803$)
0	0	1	0
1	0	2	1.61803
2	1	4	2.618021081
3	2	8	4,23603665
4	3	16	6,85403438
5	7	32	11,090033248
6	10	64	17,944006496
7	17	128	29,033940831
8	27	256	46,977787283
9	44	512	76,011469157
10	71	1024	122,98883744
11	115	2048	198,999628643
12	186	4096	321,987369134
.	.	.	.

Why exponential time complexity

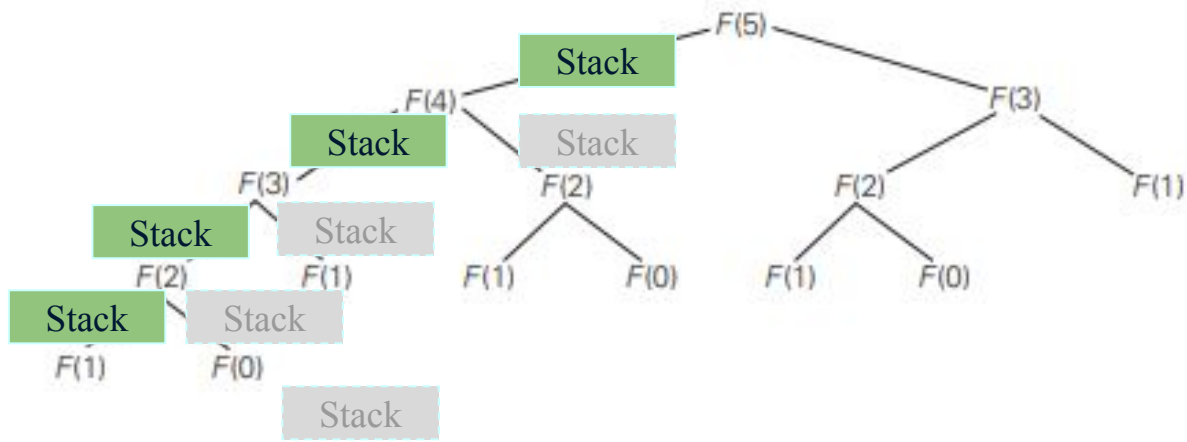
- The exponential efficiency class of the algorithm's time complexity could be anticipated by the nature of recurrence.
- It contains two recursive calls with the sizes of smaller instances only slightly smaller than size n.
- We can also see the reason behind the algorithm's efficiency by looking at a recursive tree of calls tracing the algorithm's execution.



An example of call tree for n = 5

Why linear space complexity

- The linear efficiency class of the algorithm could be anticipated by the nature of stack usage of recursive calls.
- The algorithm needs to store the local parameter values of each invocation in the stack.
- The maximum depth of such recursive calls is proportional to the value for which the fibonacci is being sought.
- Note that after each recursive call returns, the stack segment reserved for that call can be used for the next recursive call in the sequence and hence can be reused for the next call.



An example of call tree for n = 5

Much Faster Iterative implementation

ALGORITHM ***Fibonacci_iterative*****(n)**

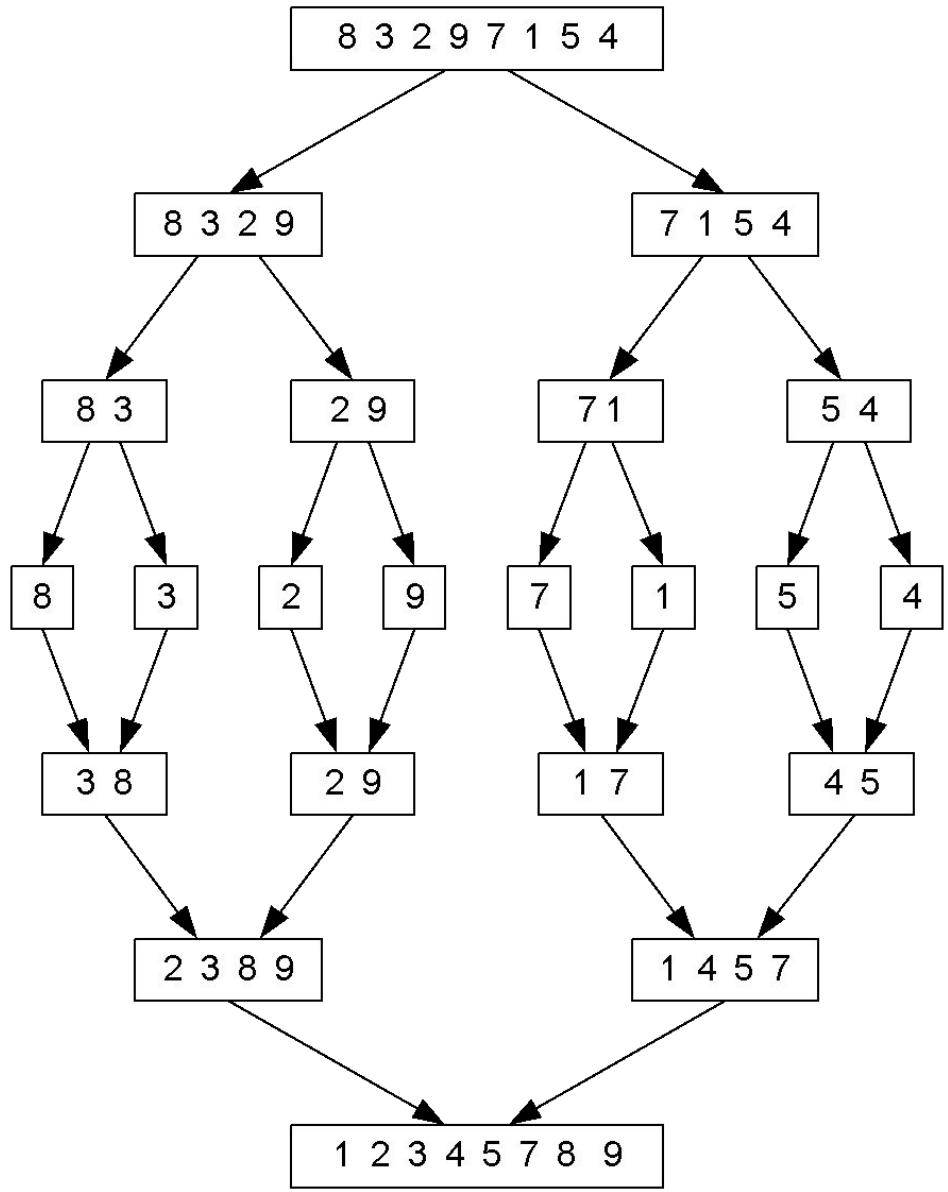
```
//Computes the n^th Fibonacci number iteratively
//Input: A nonnegative integer n
//Output: The n^th Fibonacci number.
F[0] ← 0; F[1] ←1
for i ← 2 to n do
    F[i] ←F[i-1] + F[i-2]
return F[n]
```

- Time complexity:
  - If problem size is n:
    - *O(n) i.e. linear with n*
- Space complexity:
    - *O(n) i.e. linear with n*

Example 5: Merge-sort (Divide and Conquer)

- Split array A[0..*n*-1] in two about equal halves and make copies of each half in arrays B and C.
- Sort arrays B and C recursively.
- Merge sorted arrays B and C into array A as follows:
  - Repeat the following until no elements remain in one of the arrays:
    - compare the first elements in the remaining unprocessed portions of the arrays
    - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
  - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Merge-sort Example



Pseudocode of Mergesort

ALGORITHM *Mergesort*$(A[0..n-1])$

```
//Sorts array A[0..n − 1] by recursive mergesort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
if n > 1
    copy A[0..⌊n/2⌋ − 1] to B[0..⌊n/2⌋ − 1]
    copy A[⌊n/2⌋..n − 1] to C[0..⌈n/2⌉ − 1]
    Mergesort(B[0..⌊n/2⌋ − 1])
    Mergesort(C[0..⌈n/2⌉ − 1])
    Merge(B, C, A)
```

Pseudocode of Merge

ALGORITHM $Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ j \leftarrow j+1$
    $k \leftarrow k+1$
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Analysis of Mergesort

- Selecting the basic operation as comparison of two sub array elements (the end result will not be different if we have selected the basic operation as the copying of a subarray member to the larger array, i.e., assignment. Why?)
- The recurrence relation for the number of key comparisonS

$$T(n) = 2T(n/2) + C_{merge}(n) \quad \text{for } n > 1,$$
$$T(1) = 0.$$

$C_{merge}(n)$ is number of key comparisons performed during the merging stage.

- At each step, exactly one comparison is made, after which the total number of elements in the two arrays still needing to be processed is reduced by 1.

- In the worst case, neither of the two arrays becomes empty before the other one contains just one element (e.g., smaller elements may come from the alternating arrays).

General Divide-and-Conquer Recurrence Rules

$T(n) = aT(n/b) + f(n)$ where $f(n) \in \mathrm{O}(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in \mathrm{O}(n^d)$

If $a = b^d$, $T(n) \in \mathrm{O}(n^d \log n)$

If $a > b^d$, $T(n) \in \mathrm{O}(n^{\log_b a})$

Note: The same results hold with O instead of Θ.

Examples: $T(n) = 4T(n/2) + n \Rightarrow T(n) \in \mathrm{O}(n^2)$

$T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in \mathrm{O}(n^2 \log n)$

$T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in \in \mathrm{O}(n^3)$

Analysis of Mergesort

Therefore, for the worst case;

$$C_{merge}(n) = n - 1$$

and we have the recurrence;

$T(n) = 2T(n/2) + n\text{-}1 \quad for\ n > 1,\ T(1) = 0$

If we apply the Master's theorem:

$f(n) = n\text{-}1 \Longrightarrow f(n) \in \Theta(n^1)$, so; $d=1$

$a=2,\ b=2,\ d=1,$

Since $a = b^d$

Second alternative is applicable according to Master's Theorem:

$T(n) \in O(n^1 \log n)$

T(n) ∈ O(nlogn) - Linearithmic

Analysis of Mergesort

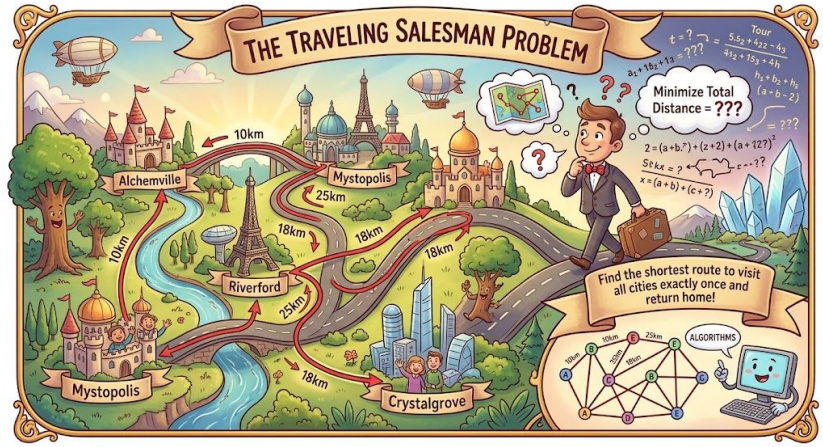
- Mergesort is linearitmic, i.e., $\Theta(n \log n)$

  Worst, Best, Average?

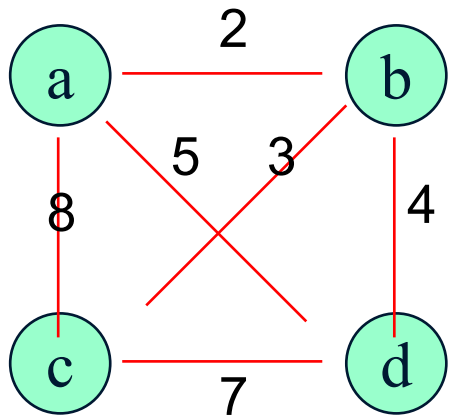
- Space complexity :
  - S(n) = n + logn
    - not in-place, uses an extra array with size equal to the size of the original.
    - The memory needs of the recursive calls also amounts to a logarithmic usage of stack segments, since the maximum depth of the recursion is equal to logn.

  - ***S(n) is O(n)***

Traveling Salesman Problem



- Given *n* cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph
- Example:



TSP by Exhaustive Search

Tour	Cost
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17
b→a→c→d→b	2+8+7+4 = 21
b→a→d→c→b	
……	

Complexity: ?

Example 6: Knapsack Problem

Given *n* items:

- **weights:** w_1 w_2 … w_n
- **values:** v_1 v_2 … v_n
- **a knapsack of capacity *W***

Find most valuable subset of the items that fit into the knapsack

Example: Knapsack capacity W=16

item	weight	value
1	2	$20
2	5	$30
3	10	$50
4	5	$10



Knapsack Problem by Exhaustive Search

Subset	Total weight	Total value
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	not feasible
{1,2,4}	12	$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

Complexity :

T(n) is $O(2^n)$